

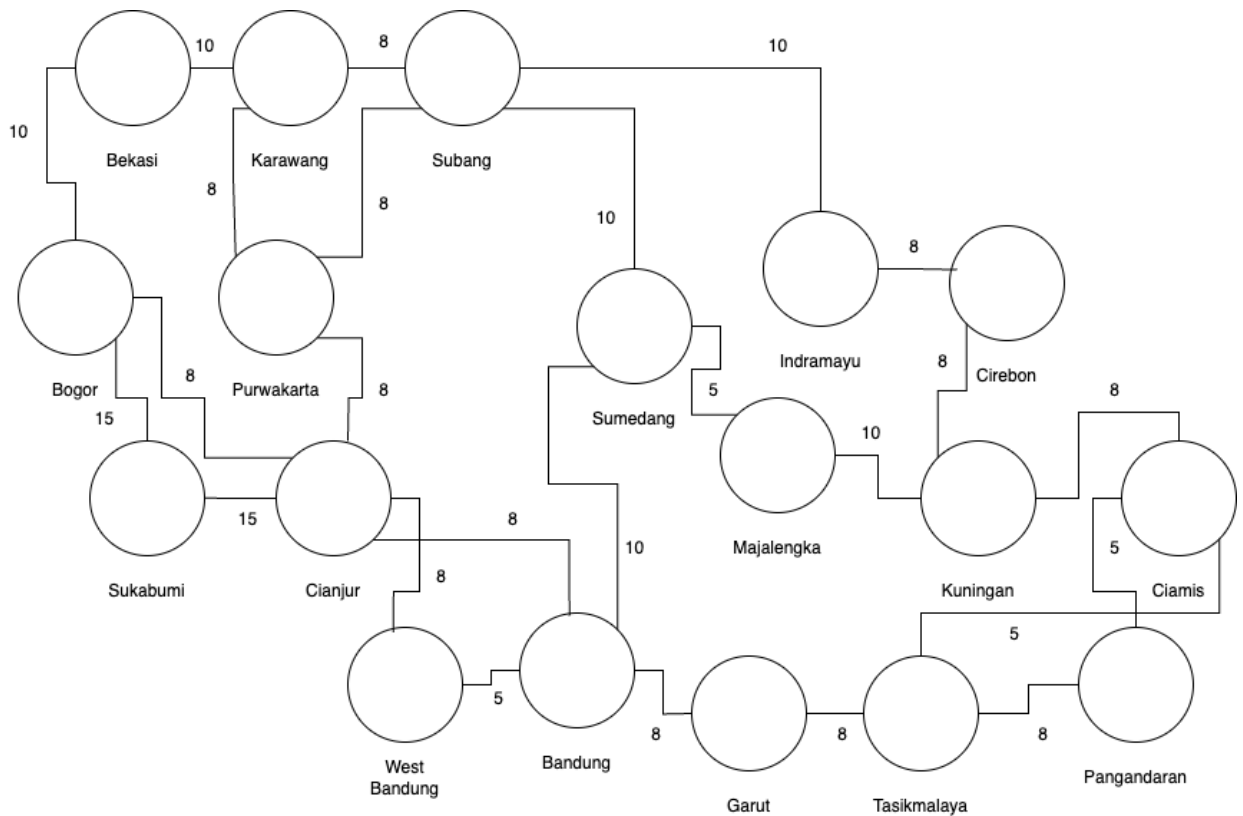
LKP 4

Informed Search Algorithm

Dengan peta yang sama dan tambahan informasi di tabel di bawah, gunakan algoritme pencarian di bawah ini secara manual, kemudian bandingkan hasil manual Anda dengan implementasi yang menggunakan AIMA Python serta ditambahkan penjelasan kode program untuk pencarian dari **Bogor** ke **Pangandaran**.

1. Greedy Search (Best First Search)
2. A-Star

Search



Lembar Kerja Praktikum Kecerdasan Buatan

Kota	Lokasi X	Lokasi Y		Kota	Lokasi X	Lokasi Y
Bekasi	10	50		Purwakarta	30	30
Karawang	20	50		Sukabumi	20	20
Subang	20	40		Cianjur	23	20
Bogor	20	30		West Bandung	25	10
Bandung	30	10		Garut	40	5
Tasikmalaya	45	5		Pangandaran	50	5
Ciamis	50	20		Kuningan	45	20
Majalengka	40	20		Cirebon	45	30
Sumedang	35	25		Indramayu	40	30

Catatan :

1. Kumpulkan LKP dalam format ZIP (Format: LKP4_NIM_Nama_Paralel.zip) yang di dalamnya terdapat laporan dalam format PDF dan file kode program dengan bahasa python dengan format nama masing-masing LKP4_NIM_Nama.pdf, LKP4_NIM_Nama.py.
2. Laporan berisi penjelasan pencarian secara **manual** dan juga penjelasan kode program, beserta hasil perbandingan keduanya.

Greedy Best-First Search (Manual)

Greedy memilih node dengan nilai heuristic terkecil:

$$f(n) = h(n)$$

Tidak mempertimbangkan biaya yang sudah ditempuh.

Perhitungan:

1. Posisi: Bogor (20,30)

$$\text{Bekasi (10,50) } h \approx \sqrt{[(50-10)^2 + (5-50)^2]} \approx 60.21$$

$$\text{Sukabumi (20,20) } h \approx \sqrt{[(50-20)^2 + (5-20)^2]} \approx 33.54$$

$$\text{Cianjur (23,20) } h \approx \sqrt{[(50-23)^2 + (5-20)^2]} \approx 30.87 \text{ (terkecil)}$$

2. Posisi: Cianjur (23,20) West Bandung (25,10) $h \approx \sqrt{[(50-25)^2 + (5-10)^2]} \approx 25.50$

$$\text{Bandung (30,10) } h \approx \sqrt{[(50-30)^2 + (5-10)^2]} \approx 20.62 \text{ (terkecil)}$$

3. Posisi: Bandung (30,10) Garut (40,5) $h \approx \sqrt{[(50-40)^2 + (5-5)^2]} \approx 10.00 \text{ (terkecil)}$

$$\text{Sumedang (35,25) } h \approx \sqrt{[(50-35)^2 + (5-25)^2]} \approx 25.00$$

4. Posisi: Garut (40,5) Tasikmalaya (45,5) $h \approx \sqrt{[(50-45)^2 + (5-5)^2]} \approx 5.00$

5. Posisi: Tasikmalaya (45,5) Pangandaran (55,5) $h \approx \sqrt{[(50-50)^2 + (5-5)^2]} \approx 0$

Hasil Greedy:

Bogor → Cianjur → Bandung → Garut → Tasikmalaya → Pangandaran

Total biaya:

$$8 + 8 + 8 + 8 + 8 = 40$$

A* Search (Manual)

$$f(n) = g(n) + h(n)$$

Dimana:

$g(n)$ = biaya yang sudah ditempuh

$h(n)$ = estimasi ke tujuan

Perhitungan:

Posisi: Bogor (20,30)

$$\approx \sqrt{[(50-20)^2 + (5-30)^2]} \approx 39.05$$

Bekasi (10,50)

$$f = 10 + 60.21 = 70.21$$

Sukabumi (20,20)

$$f = 15 + 33.54 = 48.54$$

Cianjur (23,20)

$$f = 8 + 30.87 = 38.87 \text{ (terkecil)}$$

• Posisi: Cianjur (23,20)

West Bandung (25,10)

$$f = 8 + 8 + 25.50 = 41.50$$

Bandung (30,10)

$$f = 8 + 8 + 20.62 = 36.62 \text{ (terkecil)}$$

• Posisi: Bandung (30,10)

Garut (40,5)

$$f = 16 + 8 = 24 + 10.00 = 34.00 \text{ (terkecil)}$$

Sumedang (35,25)

$$f = 16 + 10 = 26 + 25.00 = 51.00$$

• Posisi: Garut (40,5)

Tasikmalaya (45,5)

$$f = 24 + 8 + 5.00 = 37.00$$

• Posisi: Tasikmalaya (45,5)

Pangandaran (55,5)

$$f = 32 + 8 + 0 = 40$$

Jalur yang diperoleh:

Bogor → Cianjur → Bandung → Garut → Tasikmalaya → Pangandaran

Karena setiap langkah memiliki bobot 8 dan selalu mendekati tujuan, A* langsung menemukan jalur optimal.

Hasil A*: Bogor → Cianjur → Bandung → Garut → Tasikmalaya → Pangandaran

Total biaya = 40

Implementasi menggunakan AIMA Python**Greedy(BFS):**

```
# IMPORT
from aimasearch import GraphProblem, best_first_graph_search
from aimasearch import UndirectedGraph
import math

# KOORDINAT KOTA
locations = {
    'Bekasi': (10, 50),
    'Karawang': (20, 50),
    'Subang': (20, 40),
    'Bogor': (20, 30),
    'Bandung': (30, 10),
    'Tasikmalaya': (45, 5),
    'Ciamis': (50, 20),
    'Majalengka': (40, 20),
    'Sumedang': (35, 25),
    'Purwakarta': (30, 30),
    'Sukabumi': (20, 20),
    'Cianjur': (23, 20),
    'West Bandung': (25, 10),
    'Garut': (40, 5),
    'Pangandaran': (50, 5),
    'Kuningan': (45, 20),
    'Cirebon': (45, 30),
    'Indramayu': (40, 30)
}

# STRUKTUR GRAPH
west_java_graph = UndirectedGraph(dict(
    Bogor={'Cianjur': 8},
    Cianjur={'Bandung': 8},
    Bandung={'Garut': 8},
    Garut={'Tasikmalaya': 8},
    Tasikmalaya={'Pangandaran': 8},

    Bekasi={'Karawang': 10},
    Karawang={'Subang': 8, 'Purwakarta': 6},
    Subang={'Sumedang': 8},
    Sumedang={'Majalengka': 6},
    Majalengka={'Cirebon': 6, 'Kuningan': 5},
    Cirebon={'Indramayu': 7},
    Indramayu={'Subang': 9},
    Sukabumi={'Bogor': 7},
    Purwakarta={'Bandung': 7},
    Ciamis={'Tasikmalaya': 7, 'Kuningan': 6}
))
west_java_graph.locations = locations
```

```
# HEURISTIK (EUCLIDEAN)
def heuristic(node):
    x1, y1 = locations[node.state]
    x2, y2 = locations['Pangandaran']
    return math.sqrt((x1 - x2)**2 + (y1 - y2)**2)

# DEFINISI PROBLEM
problem = GraphProblem('Bogor', 'Pangandaran', west_java_graph)

# GREEDY BEST FIRST SEARCH
# f(n) = h(n)
result = best_first_graph_search(problem, heuristic)

# OUTPUT
print("Jalur Greedy:")
print(['Bogor'] + result.solution())

print("\nTotal Cost:")
print(result.path_cost)
```

Output:

```
Jalur Greedy:
['Bogor', 'Cianjur', 'Bandung', 'Garut', 'Tasikmalaya', 'Pangandaran']

Total Cost:
40
```

Program ini menggunakan algoritma Greedy Best First Search untuk mencari jalur dari Bogor ke Pangandaran. Greedy hanya menggunakan nilai heuristik $h(n)$, yaitu estimasi jarak garis lurus ke tujuan, tanpa memperhitungkan biaya yang sudah ditempuh. Akibatnya algoritma ini lebih cepat dalam memilih jalur, tetapi tidak selalu menjamin hasil yang paling optimal.

A* Search:

```
# IMPORT
from aim3.search import GraphProblem, astar_search
from aim3.search import UndirectedGraph
import math

# KOORDINAT KOTA
locations = {
    'Bekasi': (10, 50),
    'Karawang': (20, 50),
    'Subang': (20, 40),
    'Bogor': (20, 30),
```

```

    'Bandung': (30, 10),
    'Tasikmalaya': (45, 5),
    'Ciamis': (50, 20),
    'Majalengka': (40, 20),
    'Sumedang': (35, 25),
    'Purwakarta': (30, 30),
    'Sukabumi': (20, 20),
    'Cianjur': (23, 20),
    'West Bandung': (25, 10),
    'Garut': (40, 5),
    'Pangandaran': (50, 5),
    'Kuningan': (45, 20),
    'Cirebon': (45, 30),
    'Indramayu': (40, 30)
}

# STRUKTUR GRAPH
west_java_graph = UndirectedGraph(dict(
    Bogor={'Cianjur': 8},
    Cianjur={'Bandung': 8},
    Bandung={'Garut': 8},
    Garut={'Tasikmalaya': 8},
    Tasikmalaya={'Pangandaran': 8},
    Bekasi={'Karawang': 10},
    Karawang={'Subang': 8, 'Purwakarta': 6},
    Subang={'Sumedang': 8},
    Sumedang={'Majalengka': 6},
    Majalengka={'Cirebon': 6, 'Kuningan': 5},
    Cirebon={'Indramayu': 7},
    Indramayu={'Subang': 9},
    Sukabumi={'Bogor': 7},
    Purwakarta={'Bandung': 7},
    Ciamis={'Tasikmalaya': 7, 'Kuningan': 6}
))

west_java_graph.locations = locations

# HEURISTIK (EUCLIDEAN)
def euclidean_distance(node, goal):
    x1, y1 = locations[node]
    x2, y2 = locations[goal]
    return math.sqrt((x1 - x2)**2 + (y1 - y2)**2)

# DEFINISI PROBLEM
problem = GraphProblem('Bogor', 'Pangandaran', west_java_graph)

# A* SEARCH
result = astar_search(problem, lambda n: euclidean_distance(n.state,
    'Pangandaran'))

```

```
# OUTPUT
print("Jalur A*:")
print(['Bogor'] + result.solution())
print("\nTotal Cost:")
print(result.path_cost)
```

Output:

```
Jalur A*:
['Bogor', 'Cianjur', 'Bandung', 'Garut', 'Tasikmalaya', 'Pangandaran']

Total Cost:
40
```

Program ini menggunakan algoritma A* untuk mencari jalur terpendek dari Bogor ke Pangandaran pada graph Jawa Barat. A* menghitung nilai $f(n) = g(n) + h(n)$, yaitu biaya yang sudah ditempuh ditambah estimasi jarak ke tujuan menggunakan rumus Euclidean. Karena mempertimbangkan biaya nyata dan perkiraan jarak, algoritma ini mampu menemukan jalur optimal dengan total cost paling kecil.

Perbandingan Greedy (Best First Search) vs A* Search

1. Rumus
Greedy: $f(n) = h(n)$ (Hanya mempertimbangkan estimasi jarak ke tujuan)
A*: $f(n) = g(n) + h(n)$ (Mempertimbangkan biaya yang sudah ditempuh + estimasi jarak)
2. Pertimbangan Biaya
Greedy: Tidak memperhitungkan biaya perjalanan yang sudah dilewati.
A*: Memperhitungkan total biaya dari awal sampai node saat ini.
3. Optimalitas
Greedy: Tidak selalu menghasilkan jalur terpendek.
A*: Menjamin jalur terpendek jika heuristik bersifat admissible (tidak melebihi biaya sebenarnya).
4. Kecepatan
Greedy: Biasanya lebih cepat karena hanya melihat heuristik.
A*: Sedikit lebih lambat karena menghitung $g(n)$ dan $h(n)$.
5. Ketepatan Hasil
Greedy: Bisa terjebak pada jalur yang terlihat dekat tetapi sebenarnya mahal.
A*: Lebih stabil dan akurat dalam menemukan solusi terbaik.

Kesimpulan

Program yang dibuat menunjukkan bahwa baik Greedy Best First Search maupun A* Search dapat menemukan jalur dari Bogor ke Pangandaran dengan total biaya 40 pada struktur graph yang digunakan. Namun secara teori, A* lebih unggul karena mempertimbangkan biaya aktual dan estimasi jarak sehingga lebih menjamin solusi optimal dibandingkan Greedy.